



Draw It or Lose It
CS 230 Project Software Design Template
Version 1.0

Table of Contents

CS 230 Project Software Design Template	1
Table of Contents	2
Document Revision History	2
Executive Summary	3
Requirements	3
Design Constraints	3
System Architecture View	3
Domain Model	3
Evaluation	4
Recommendations	5

Document Revision History

Version	Date	Author	Comments
1.0	06/07/2026	Mel Toribio Blanco	Initial design for Draw It or Lose It web-based game platform

Instructions

Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

The Gaming Room has a successful television game show called “Draw It or Lose It” and wants to expand it into a web-based game that can support multiple platforms. The new application needs to manage games, teams, and players consistently to prevent any duplicate game names. The goal is to be accessible from different client devices. To keep the code organized and easy to maintain, the design will use object-oriented principles (inheritance, composition, singleton pattern, etc.). The design will also have some client requirements and needs to be user-friendly for it to successfully execute. Overall, this design provides a flexible, scalable, and secure foundation for The Gaming Room application. The architectural decisions documented here support current requirements while allowing for future expansion, additional platforms, and increased user demand without requiring significant redesign.

Requirements

- Multiplayer gameplay for Draw It or Lose It
- Support multiple simultaneous games, each with unique name
- Provide consistent access to game data across all clients
- Operate within a web based distributed environment to support different platforms (desktop + mobile)
- Time limitations
 - “A game consists of four rounds of play lasting one minute each. Drawings are rendered at a steady rate and are fully complete at the 30-second mark. If the team does not guess the puzzle before time expires, the remaining teams have an opportunity to offer one guess each to solve the puzzle with a 15-second time limit.”
- Authentication must verify user identity before allowing access.
- Authorization must restrict access to resources based on user permissions.
- Data transmitted between the client and server must be protected.
- The system must be reliable and remain stable during extended use.
- The application should be maintainable and easy to update or expand

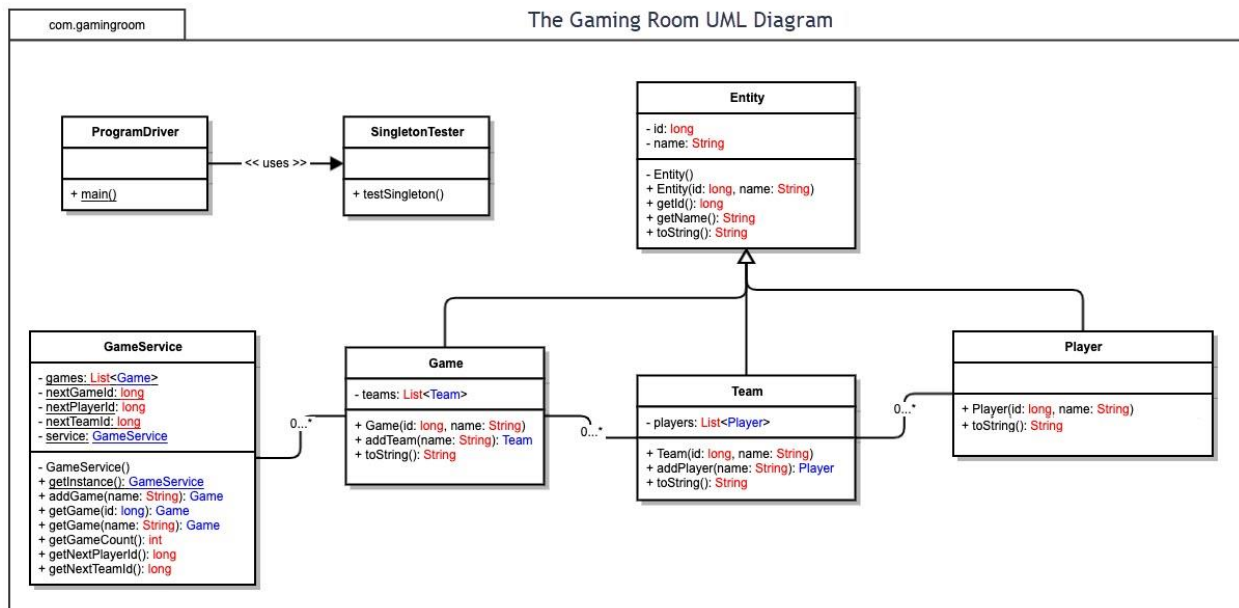
Design Constraints

- Since its web-based distributed environment, there are constraints in network communication, security, and responsiveness with other browsers and devices.
- Having multiple games with unique names is needed for the best user experience. They need to have different names all through out to prevent errors
- We must design the game, knowing that only one copy of the game service can run in memory at once.
- The application must support multiple operating platforms.
- A client–server architecture must be used to enable cross-platform compatibility.
- The server must support Java-based development.
- Industry-standard design patterns must be used.
- The system must follow professional software development best practices.

System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model



UML Diagram Description

Object Oriented Principles Shown:

Inheritance:

- Games, Team, and Player all inherit from Entity.
- Common attributes (Id, name) and methods shared across subclasses

Composition:

- A Game contains multiple team objects
- A team contains multiple player objects

Singleton pattern:

- Game Service has a private constructor
- GetInstance() ensures only one instance manages all games

Encapsulation:

- Each class stores and protects its own data fields

Iterator Pattern:

- Used in GameService to loop through collections of games.
- Allows the system to search for existing games, teams, and players without exposing the underlying collection structure.
- Helps prevent duplicate objects from being created.

Class Details

The UML class diagram for the Gaming Room shows the core game objects and how they are managed. At the top is the entity class which defines shared attributes and behavior for things that have an Id and a name. The game, team, and player classes are all inherited from the entity. The GameService class is responsible for creating and managing game objects. It maintains a list of Game instances and tracks the next available IDs for games, teams, and players. This class has a Singleton pattern.

Each Game object contains a collection of Team objects, and each team contains a collection of player objects. This is where composition/aggregation is shown. Where larger objects are built from collections of smaller objects. The "0..*" indicates that a game can have many teams and a team can have many players. The diagram also shows how driver classes like ProgramDriver and Singleton Tester use GameService to exercise and test the design. All these concepts demonstrated in the UML diagram illustrate the concept of the code and make it modular and extensible.

Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all must work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	Provides a stable UNIX-based platform with good development tooling. Hostin cost is higher, and it has limited cloud availability. Best for local development	Open source, fast, secure, and highly scalable. Perfect for cloud deployment. Linux offers a cost-effective and reliable server environment.	It integrates well with other Microsoft technologies. Higher resource usage than Linux. However, licensing costs are typically higher than Linux	Phones and tablets have limited processing power, memory, storage, and network reliability. Not suitable as a hosting platform.
Client Side	Accessible through a standard web browser (safari, chrome, etc.). Development costs are moderate. The main consideration is a responsive UI design.	Accessible through a web browser (chrome/Firefox). There is little Linux specific client code to maintain. Priority is making sure browser-based features work properly across different Linux distributions and screen resolutions	Most common client platforms. Licensing costs are higher. Works fully through browser	must be responsive and touch friendly so it works well on different screen sizes. Goal is to optimize network request for slower or unstable mobile connections. If future native features are desired, additional time and expertise are required to build and maintain.
Development Tools	Supports java/JavaScript. IDEs eclipse, VScode, and IntelliJ.	Works seamlessly with Java and web technologies.	Supports Java JDK, eclipse, and Git. Is beginner friendly. Works well but different slightly from Linux deployment environment	If building native mobile appts, tools like Xcode and Android stuffie would be required

Recommendations

Analyze the characteristics of techniques specific to various systems architectures and recommend The Gaming Room. Specifically, address the following:

1. **Operating Platform:** Implementing a web-based operating platform is recommended when hosting Draw It or Lose It. It will allow it to be used on a variety of different platforms and devices, elevating its audience and reaching more users while maintaining a consistent UX. Linux would provide impressive performance.
2. **Operating Systems Architectures:** Use a multi-tier architecture as it handles HTTP requests. Application tier runs game logic and java services. Database tiers store game, team, and player data.
3. **Storage Management:** Use a relational database (MySQL) to centralize shared storage, making it more consistent. This can be hosted on the same Linux platform or managed cloud database service. Regular backups, access controls, and transaction support are important storage management concerns for protecting this data.
4. **Memory Management:** On the Linux platform, JVM handles automatic memory management via garbage collection. The GameService singleton manages in-memory collections of games, teams, and players during runtime. While JVM reclaims memory for objects that are no longer needed. Sizing the JVM heap and monitoring resource usage will be recommended to avoid memory leaks and maintain responsiveness.
5. **Distributed Systems and Networks:** Clients communicated Using HTTP/HTTPS. Use JSON or HTML for lightweight communication. Add load balancers for horizontal scaling. Build in error handling for latency and dropped connections.
6. **Security:** Security is critical for protecting both the players and the Gaming Room's data. Requires HTTPS for encrypted communication. The application should implement authentication and authorization to protect resources. Securely store any sensitive data (password hashing, restricted DB access). The Linux platform supports firewalls, user and group permissions, and regular patching which all contribute to a secure deployment environment.